Government College University Faisalabad

Department of Information Technology

Student Registration App

CIT-646 Mobile Application Development - Assignment 2 Report



Item	Detail
Name / Roll No	Hasher Amin / 245423
Class	BS (IT), 5th semester, Evening-A
Instructor	Dr. Afzaal Hussain
Due date	Monday, 2 November 2026, 10:50 AM
Technology	Flutter (Dart), Material 3

1. Introduction

This report describes how the Student Registration App was built. The app is one scrolling screen where a student fills in a form (name, email, age, passwords, programme, semester, courses and some options) and presses Register. The form is checked in a fixed order, and a valid registration is saved in a small backend class. Each saved student then appears as a Card in a list below the form.

The app follows the assignment exactly: the same 16 views in the same order, the same validation messages, and the same helper functions. Only the widgets taught in the lecture are used, and only if/else is used for decisions (no ? : operator).

2. Project structure

The data (Student and StudentStore) is kept separate from the screen (main.dart), as the assignment asks.

```
HasherAmin(245423) - Assignment2/
├── pubspec.yaml                (dependencies + asset list)
├── assets/
│   └── images/
│       └── gcuf.jpg            (banner shown at the top)
└── lib/
    ├── main.dart               (RegisterPage: screen, validation, state)
    ├── student.dart            (Student class: initials(), summary())
    └── student_store.dart      (StudentStore: add(), all(), count())
```

File	Purpose
main.dart	Starts the app and contains RegisterPage, a StatefulWidget with all views, helper functions and state.
student.dart	Holds one registration (8 fields) and two methods that build the avatar initials

File	Purpose
	and the summary text.
student_store.dart	Keeps all students in a private list in memory and refuses a repeated email.
pubspec.yaml	Lists the Flutter SDK dependency and registers assets/images/ so Image.asset can load the banner.
gcuf.jpg	The banner image (banner shown at the top)

3. Task C - the backend classes

3.1 Student (student.dart)

Student is a plain class with eight final fields: name, email, age, programme, semester, courses, updates and color. All are required named parameters, so a student cannot be created half-empty.

- `initials()` splits the name on spaces, ignores empty pieces, takes the first letter of the first two words in capitals, and returns `'?'` when the result is empty.
- `summary()` returns text like `'BS IT, 5th semester, 2 courses, updates ON'`. An if/else writes "1 course" for one course and "n courses" otherwise, and another if/else writes ON or OFF.

3.2 StudentStore (student_store.dart)

- `emailExists(email)` loops over the private list with a for loop and compares emails in lower case, so Ali@GCUF.edu.pk equals ali@gcuf.edu.pk.
- `add(student)` returns "This email is already registered" if the email exists. Otherwise it adds the student and returns "Registered successfully".
- `all()` returns the list and `count()` returns its length. These two feed the student list and the "Students (n)" heading.

The students live only in memory while the app is running. Saving to a database is planned for a later week, so nothing is written to the phone.

4. Task A - designing the screen

RegisterPage is a StatefulWidget because the screen changes while the user types and taps. The body is a SingleChildScrollView containing Padding(16) and a Column. The views are placed in this order, with SizedBox gaps between groups.

#	View	Widget(s)	How it is built
1	Banner	ClipRRect + Image.asset	Height 84, BoxFit.cover, corner radius 12.
2	Avatar	CircleAvatar	Radius 36; shows the initials or ?.
3	Colour circles	GestureDetector + CircleAvatar	Teal, indigo, orange written out; each onTap calls _pickColor().
4	Name	TextField	Controller, person icon, onChanged updates the initials.
5	Email	TextField	TextInputType.emailAddress, email icon.
6	Age	TextField	TextInputType.number.
7	Password	TextField + IconButton	obscureText: _hide; eye icon calls _toggleHide().
8	Confirm password	TextField	obscureText: true.

#	View	Widget(s)	How it is built
9	Programme	RadioGroup + RadioListTile	BS IT, BS CS, BS SE.
10	Semester	DropdownButton	1st to 8th, hint “Select semester”, isExpanded.
11	Courses	CheckboxListTile × 4	Four course names, ticked state in a bool list.
12	Updates	SwitchListTile	“Email me updates”, on by default.
13	Rules	CheckboxListTile	“I accept the rules”, after a Divider.
14	Buttons	Row + Expanded	Register (ElevatedButton) and Reset (OutlinedButton), 10-pixel gap.
15	Message	Text	Shows _message in _messageColor.
16	Student list	Card + ListTile	Heading “Students (n)”, then one Card per student.

The AppBar title is “Student Registration” and it shows the number of saved students in a small circle on the right, as in the sample app.

5. State and helper functions

All changing values are fields of _RegisterPageState: the five TextEditingControllers, _hide, _color, _programme, _semester, _ticked (four bools), _updates, _rules, _message and _messageColor. A value is changed only inside setState(), so Flutter redraws the screen. The controllers are disposed in dispose().

Function	Returns	What it does
_makeInitials(name)	String	First letters of the first two words in capitals; ? if empty.
_pickColor(c)	void	Stores the chosen colour inside setState().
_toggleHide() / _eyeIcon()	void / IconData	Show or hide the password; pick the matching eye icon.
_isValidEmail(email)	bool	No spaces, text before @, a dot after @ with text between, text after the dot.
_hasDigit(text)	bool	true if any character is 0–9 (checked in a for loop).
_checkForm()	String?	Runs rules 1–13 in order and returns the first message, or null.
_registerAction()	VoidCallback?	Returns _register when the rules box is ticked, otherwise null.
_register()	void	Calls _checkForm(); if null, builds a Student and calls _store.add().
_reset()	void	Clears the controllers and every choice; keeps the saved students.

5.1 The disabled Register button

The button is written as onPressed: _registerAction() (with brackets, because the function must be called to get the answer). When the rules box is not ticked the function returns null, and Flutter shows the button in grey and ignores taps. When it is ticked, the function returns _register without brackets, so the button runs _register only when it is pressed.

```
VoidCallback? _registerAction() {
  if (_rules) {
    return _register;
  } else {
    return null;
  }
}
```

5.2 Avatar initials on every key press

The name TextField has onChanged: _onNameChanged, which only calls setState(() {}). The rebuild runs _makeInitials(_nameController.text) again, so the CircleAvatar text changes on every key press.

6. Task B - validation

When Register is pressed, _checkForm() tests the fields in this order and stops at the first problem. The messages are exactly those given in the assignment. The text of the name, email and age is trimmed first; the passwords are not trimmed.

Order	Field	Rule	Message
1	Name	Not empty after trim()	Please enter your name
2	Name	At least 3 letters	Name must be at least 3 letters
3	Email	Not empty	Please enter your email
4	Email	_isValidEmail()	Please enter a valid email
5	Age	int.tryParse() is not null	Age must be a number
6	Age	16 to 60	Age must be between 16 and 60
7	Password	At least 6 characters	Password must be at least 6 characters
8	Password	Has a digit	Password must contain a number
9	Confirm	Same as password	Passwords do not match
10	Programme	_programme is not null	Please choose a programme
11	Semester	_semester is not null	Please choose a semester
12	Courses	At least one ticked	Select at least one course
13	Courses	At most three ticked	Select at most three courses
14	Backend	_store.add() refuses a repeated email	This email is already registered

Rules 1–13 are inside _checkForm(). Rule 14 is decided by the store inside _register(). If every check passes and the store accepts the student, the message “Registered successfully” is shown in green; every problem message is shown in red.

6.1 How _register() works

```
String? problem = _checkForm();
if (problem != null) { show problem in red; return; }

// build the chosen courses with if statements
Student student = Student(name: ..., programme: _programme!, ...);
String result = _store.add(student);
// green if result == 'Registered successfully', otherwise red
```

The exclamation mark in _programme! and _semester! is safe because _checkForm() has already made sure they are not null.

7. Task D - list and Reset

The heading “Students (n)” uses _store.count(). The helper _studentCards() loops over _store.all() with a for loop and returns one Card with a ListTile for each student. The leading CircleAvatar uses the student’s colour and initials, the title is the name, and the subtitle is summary(). The list is added to the Column with the spread operator (...).

Reset clears all five controllers with controller.clear(), unticks every box (including the rules box), sets programme and semester back to null, turns updates back on, hides the password again, returns the avatar to teal (with ? as the initials, because the name is empty), and clears the message. The saved students are not touched, because they belong to the store and not to the form.

8. How to run the app

Flutter 3.35 or newer is needed because the programme buttons use RadioGroup. Check the version with flutter --version.

```
1.  flutter create student_registration
2.  cd student_registration
3.  Delete the file test/widget_test.dart
4.  Replace lib/ with the three files of this project (main.dart,
    student.dart, student_store.dart)
5.  Copy the folder assets/ (with images/gcuf.jpg) into the project
6.  Replace pubspec.yaml with the one from this project
7.  flutter pub get
8.  flutter run          (connect a phone or start an emulator first)
9.  flutter analyze      (should show no warnings)
```

If the Flutter version is older than 3.35, the RadioListTile widgets need groupValue and onChanged on each tile instead of the RadioGroup wrapper, as noted in the assignment.

9. Testing

The following checks were planned from the assignment tips and should be repeated on the device before submitting, taking the five screenshots at the same time.

Test	Expected result
Open the app and look at Register	Button is grey until “I accept the rules” is ticked
Type “ali@gcuf” as email and press Register	Please enter a valid email (red)
Password “flutter” (no digit)	Password must contain a number
Fill everything correctly and press Register	Registered successfully (green) and a Card appears
Register again with ALI@gcuf.edu.pk	This email is already registered
Press Reset	Form is empty, avatar shows ? in teal, list is kept

10. Conclusion

The app uses the widgets and ideas from the lecture: StatefulWidget and setState(), controllers, ElevatedButton and OutlinedButton, radio, dropdown, checkbox and switch inputs, CircleAvatar with GestureDetector, and Card with ListTile. Keeping Student and StudentStore in their own files shows

the idea of separating data from the screen, and the helper functions keep build() short and readable.